

# **INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA**

**Department of Computer Science Engineering**

**Module 2 — Assingment and Project Report**

**Object Detection System Analysis**

**Submitted by: Arghya Dutta**

**Enrolment Number: 12023002001201**

**Class Roll Number: C\_79**

# ASSINGMENT

Evaluate and compare object detection models (YOLO, SSD, Faster R-CNN) on accuracy, speed, and suitability for real-time use in AVs.

---

## Evaluation and Comparison of YOLO, SSD, and Faster R-CNN for Autonomous Vehicles

Object detection is a key component of an autonomous vehicle (AV) perception system because it must identify vehicles, pedestrians, traffic signs, cyclists, and other road objects with both **high accuracy and low latency**. YOLO, SSD, and Faster R-CNN represent different approaches to this problem.

| Feature                     | YOLO                               | SSD                          | FASTER R-CNN                               |
|-----------------------------|------------------------------------|------------------------------|--------------------------------------------|
| Architecture                | One-stage detector                 | One-stage detector           | Two-stage detector                         |
| Accuracy                    | High                               | Moderate to High             | Generally very high                        |
| Detection Speed             | Very high                          | High                         | Lower                                      |
| Real-time capability        | Excellent                          | Good                         | Limited on resource – constrained hardware |
| Small-object detection      | Good                               | Moderate                     | Strong                                     |
| Computational requirement   | Low to high                        | Low to moderate              | High                                       |
| Latency                     | Low                                | Low                          | Higher                                     |
| AV suitability              | Excellent for real-time perception | Good for lightweight systems | Useful where accuracy is prioritized       |
| Deployment on edge hardware | Very suitable                      | Suitable                     | More difficult                             |

### 1. YOLO

**YOLO (You Only Look Once)** performs object detection in a single network pass. This one-stage design gives it very low inference latency and makes it particularly suitable for processing continuous camera streams.

Modern YOLO models provide several model sizes so that accuracy can be traded against computational cost. For example, current YOLO26 models report **40.9–57.5 mAP on COCO**, with reported TensorRT inference times of approximately **1.7–11.8 ms** depending on model size and hardware. ([Ultralytics Docs](#))

**Advantages for AVs:**

- Very fast inference
- Low latency
- Good accuracy
- Multiple lightweight versions for edge deployment
- Suitable for detecting objects continuously from vehicle cameras
- Can be combined with tracking and speed-estimation pipelines ([Ultralytics Docs](#))

**Limitation:** Smaller/faster YOLO variants can sacrifice some accuracy, particularly for difficult or very small objects.

## 2. SSD

**SSD (Single Shot MultiBox Detector)** is also a one-stage detector. It predicts object classes and bounding boxes from multiple feature maps at different scales.

Its major advantage is its relatively good compromise between **speed, model complexity, and accuracy**. SSD can therefore be useful in embedded or computationally constrained systems.

However, SSD generally has less accuracy than stronger modern detectors, particularly when objects are small, distant, heavily occluded, or appear in complex scenes.

### Advantages for AVs:

- Fast inference
- Relatively lightweight
- Easier to deploy on limited hardware
- Suitable for applications where moderate detection accuracy is acceptable

**Limitation:** Small and distant objects can be more difficult to detect reliably compared with stronger architectures.

## 3. Faster R-CNN

**Faster R-CNN** is a **two-stage detector**. First, a Region Proposal Network identifies possible object regions, and then the detector classifies and refines those regions.

This additional processing generally provides strong localization and detection accuracy. Recent autonomous-driving research comparing YOLOv5 and Faster R-CNN found that YOLOv5 had advantages in overall mAP, recall, and training efficiency in the tested scenarios, while Faster R-CNN showed advantages for **small, distant objects and challenging lighting conditions**. ([arXiv](#))

### Advantages for AVs:

- High detection accuracy
- Strong localization
- Good performance on small/distant objects
- Useful when detection quality is prioritized over latency

### Limitations:

- Higher computational requirements
- Greater inference latency
- Less suitable for high-frame-rate perception on constrained hardware

---

## Accuracy vs Speed

The fundamental trade-off can be represented as:

**Faster R-CNN → Accuracy/Localization ↑, Speed ↓**

**SSD → Balanced computational efficiency**

**YOLO → Speed ↑ + Accuracy ↑**

The exact ranking depends heavily on the particular model version, training dataset, image resolution, hardware, and optimization method. Therefore, raw FPS or mAP values from different papers should not be treated as directly interchangeable.

For example, current YOLO benchmarks demonstrate that increasing model size generally increases COCO mAP while also increasing inference latency. ([Ultralytics Docs](#))

## Suitability for Autonomous Vehicles

For an AV, detection must happen quickly enough for the perception system to react to changing road conditions. **Latency is therefore as important as accuracy**. A highly

accurate detector that takes too long to process each frame may not be appropriate for real-time driving.

A practical comparison is:

- **YOLO:** Best suited to real-time camera-based perception where low latency is essential.
- **SSD:** Suitable for lightweight or computationally constrained AV/ADAS systems.
- **Faster R-CNN:** Suitable when high detection quality is more important than real-time throughput, such as offline analysis or computationally powerful perception systems.

## Conclusion

**YOLO provides the strongest overall balance of detection accuracy and inference speed for real-time AV applications.** SSD offers a lightweight alternative with good speed, while Faster R-CNN provides strong detection and localization capability at the cost of greater computational requirements and latency.

For an autonomous vehicle operating with live camera feeds, the final choice should be validated on the **actual target hardware and driving dataset**, measuring mAP, precision, recall, FPS/latency, memory usage, and performance under different lighting and traffic conditions. Modern YOLO tooling also supports benchmarking across deployment formats such as ONNX and TensorRT, which is useful for this hardware-specific evaluation. ([Ultralytics Docs](#))

### 1. Introduction

Automotive systems contain several important components that need to be correctly identified during vehicle inspection, maintenance and monitoring. Manual identification of these components can be time consuming and may depend on the experience of the person performing the inspection. Computer vision and deep learning can help automate this process by detecting and classifying components directly from images or video.

This project presents an Automotive Engine Component Detection System using YOLOv8. The main objective is to develop a deep learning based object detection model capable of locating different automotive engine components and identifying their classes. Unlike simple image classification, object detection provides both the class of the object and its location through bounding boxes.

The trained model is also integrated with OpenCV and a laptop webcam to demonstrate real time detection. This makes the project relevant to applications such as automated vehicle inspection, engine component monitoring, maintenance assistance and intelligent automotive systems.

### 2. Dataset and Technologies Used

The project uses the Automotive Engine Part Detection dataset in YOLOv8 format. According to the dataset configuration, it contains 2,407 images and 9 object classes. The classes are Air intake, Battery, Brake oil, Collant, Fuse Box, Main Engine, Oil cap, Radiator and Sprinkler. The dataset is organized into training, validation and testing directories, with corresponding image and label files.

The dataset was processed using Python and the YOLO compatible annotation format. Sample images were also inspected to understand the appearance and distribution of the different automotive components.

The major technologies used in the project are Python, Ultralytics YOLOv8, PyTorch, OpenCV, Matplotlib and Google Colab. GPU acceleration was enabled during training, and the project used an NVIDIA Tesla T4 GPU. OpenCV was used separately for accessing the webcam, processing video frames and displaying the detection results.

### 3. Model Training and Implementation

A pretrained YOLOv8n model was selected as the base model and fine tuned using the automotive component dataset. Using a pretrained model helps the detector start with

previously learned visual features and then adapt them to the specific automotive components in the dataset.

The model was trained for 30 epochs with an image size of 640 × 640 pixels and a batch size of 16. Training was performed using the Tesla T4 GPU available in Google Colab. The YOLOv8n architecture was selected because it provides a relatively lightweight solution while maintaining strong object detection performance.

During training, the model learned to recognize the visual characteristics of the nine component categories and predict their locations using bounding boxes. The final trained model was saved as final\_automotive\_yolo\_best.pt. This trained weight file was then used for validation and real time inference.

The overall implementation consists of several stages: dataset preparation, model initialization, model training, validation, performance evaluation and deployment for real time detection.

## 4. Model Performance and Results

The trained YOLOv8 model demonstrated strong performance on the validation dataset. The validation process contained 450 images with 1,713 object instances. The major evaluation metrics obtained were:

| Metric        | Result |
|---------------|--------|
| Precision     | 93.42% |
| Recall        | 92.09% |
| mAP@0.50      | 95.57% |
| mAP@0.50:0.95 | 81.72% |

Precision of 93.42% indicates that a high proportion of the objects predicted by the model were correctly identified. Recall of 92.09% indicates that the model was able to detect most of the relevant objects present in the validation images.

The mAP@0.50 value of 95.57% shows strong overall detection performance when an Intersection over Union threshold of 0.50 is used. The mAP@0.50:0.95 value of 81.72% provides a stricter evaluation across multiple IoU thresholds and gives an indication of the quality of the predicted bounding boxes.

The results demonstrate that the trained YOLOv8 model can effectively distinguish between the different automotive component classes while also accurately locating them within an image.

## 5. Real Time Detection Using OpenCV

After training and validation, the model was integrated with OpenCV to perform real time detection using a laptop webcam. The webcam continuously captures video frames, and each frame is passed to the trained YOLOv8 model for inference.

A confidence threshold of 0.40 was used during inference, meaning that predictions below this confidence level were filtered out. The input image size was maintained at 640 pixels. YOLOv8 processes each frame and returns the detected objects, their bounding boxes, class labels and confidence values.

The detected results are drawn directly onto the video frame using the YOLO plotting function. The system also calculates the FPS (Frames Per Second) to provide an indication of the real time processing performance.

The main inference section is:

```
model = YOLO("final_automotive_yolo_best.pt")
```

```
cap = cv2.VideoCapture(0)
```

```
results = model(
```

```
    frame,
```

```
    conf=0.40,
```

```
    imgsz=640,
```

```
    verbose=False
```

```
)
```

The live video is displayed through an OpenCV window titled “Automotive Component Detection”. The user can terminate the program by pressing Q. This implementation demonstrates how the trained model can move beyond static image detection and operate on continuously captured video.

## 6. Conclusion and Future Scope

The project successfully developed an automotive engine component detection system using YOLOv8. The model was trained on a dataset containing 2,407 images and nine automotive component classes. After training, the system achieved 93.42% precision, 92.09% recall, 95.57% mAP@0.50 and 81.72% mAP@0.50:0.95 on the reported validation results.

The integration of the trained model with OpenCV further demonstrated its ability to perform object detection on live webcam input. The system can identify components,

display their locations using bounding boxes and provide confidence information while processing video frames.

The project can be extended in several directions. Future development could include integration with vehicle inspection cameras, CCTV systems or automotive service platforms. Additional datasets could be used to improve performance under different lighting conditions, camera angles and component appearances. The system could also be extended from component detection to automated defect detection, damaged component identification and intelligent vehicle maintenance assistance.

## 7. PROJECT LINK

[https://github.com/arghyadutta7878alt/automotive\\_object\\_detection](https://github.com/arghyadutta7878alt/automotive_object_detection)